

Video Synchronization

Cross-device video synchronization for the Lightning TV app — how the prototype works, where it falls short, and what a production version needs from the backend.

Working prototype. Not production-ready.

This is a working prototype: the sync algorithm, drift correction, and crash-recovery behavior have all been built and tested, including live runs across two Android TV emulators and a forced device restart. It is not production-ready — it runs one hardcoded video through a single unauthenticated Node process with no persistence, scheduling, or admin controls. Sections 1–5 explain how the mechanism works. Section 6 lists what it doesn't solve. Section 7 lays out what a production backend would need to add.

1. The mental model

The prototype doesn't behave like an on-demand player that remembers where you paused. It behaves like a broadcast channel: a video is always somewhere in its runtime according to a shared clock, and every device that tunes in — now or ten minutes from now — lands wherever that clock currently says the show is.

One small process answers "what time is it in the video": `sync-server1.cjs`, which hands out a fixed start time. Every client computes its own position from that start time and its own local clock — nothing about playback position is ever pushed to a device or stored per-device. Kill a client and bring it back, and it works out the live position from scratch.

2. System overview

Three kinds of actor, one shared fact between them:

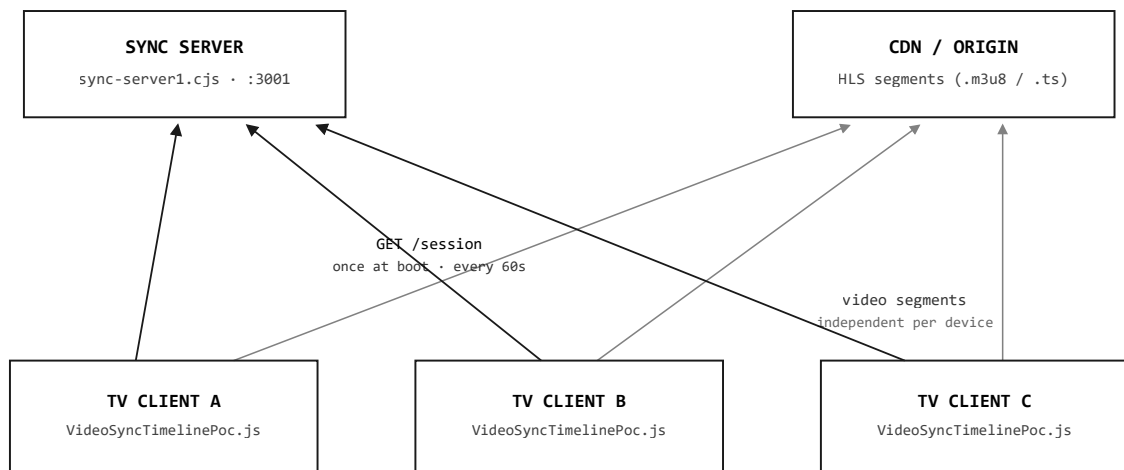


Fig. 1 — Every client resolves the shared clock through the sync server, then streams independently from the CDN. TVs never open a connection to each other — no peer link, no leader device, only a common reference point.

3. Client lifecycle

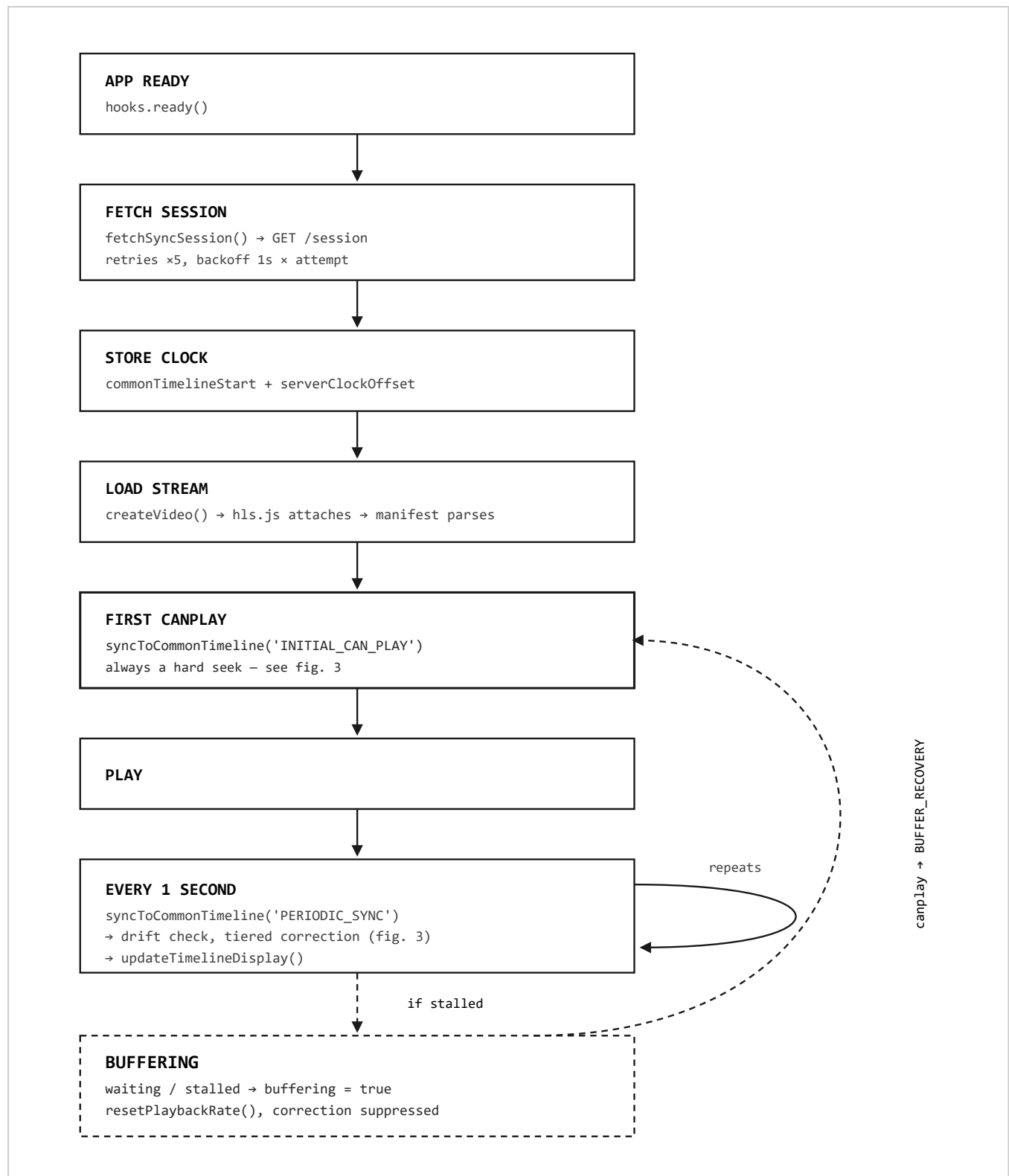


Fig. 2 — The steady state is the boxed loop at the bottom: a 1-second tick that checks drift and corrects it. A stall doesn't break the loop — it flips `buffering` on, which the correction step reads and backs off from until the player reports `canplay` again.

4. Drift correction

Every tick computes `drift = video.currentTime - targetPosition`. What happens next depends on how far off that drift is.

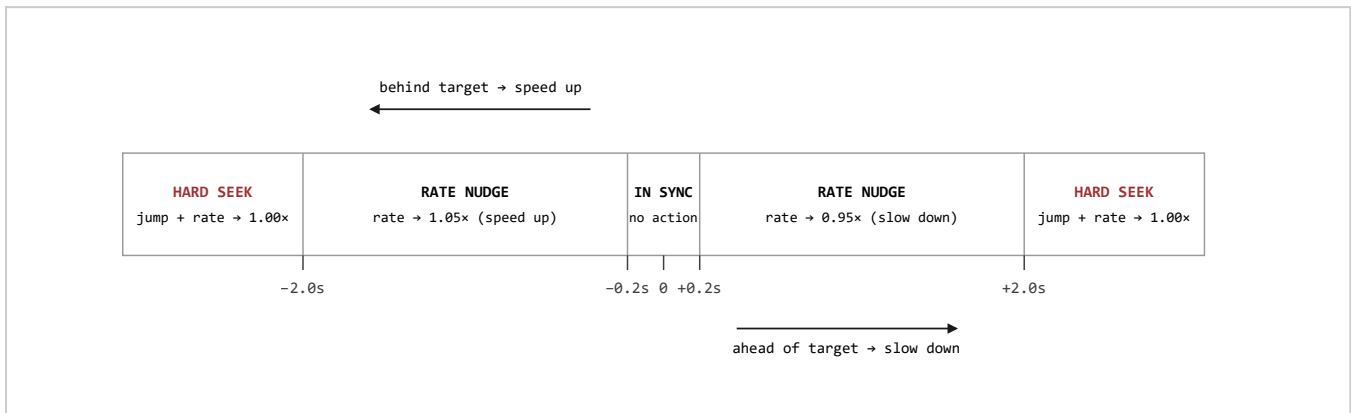


Fig. 3 — Only the outer bands seek. Everything inside $\pm 2s$ is absorbed by nudging `video.playbackRate`, which never touches the HLS buffer — this is what stopped the original seek-loop buffering bug.

5. Recovering after a device drops

Tested directly: one TV was force-stopped (simulating a crash or power cycle) while a second kept playing. After roughly 37 seconds down, the first was relaunched and logged back in.

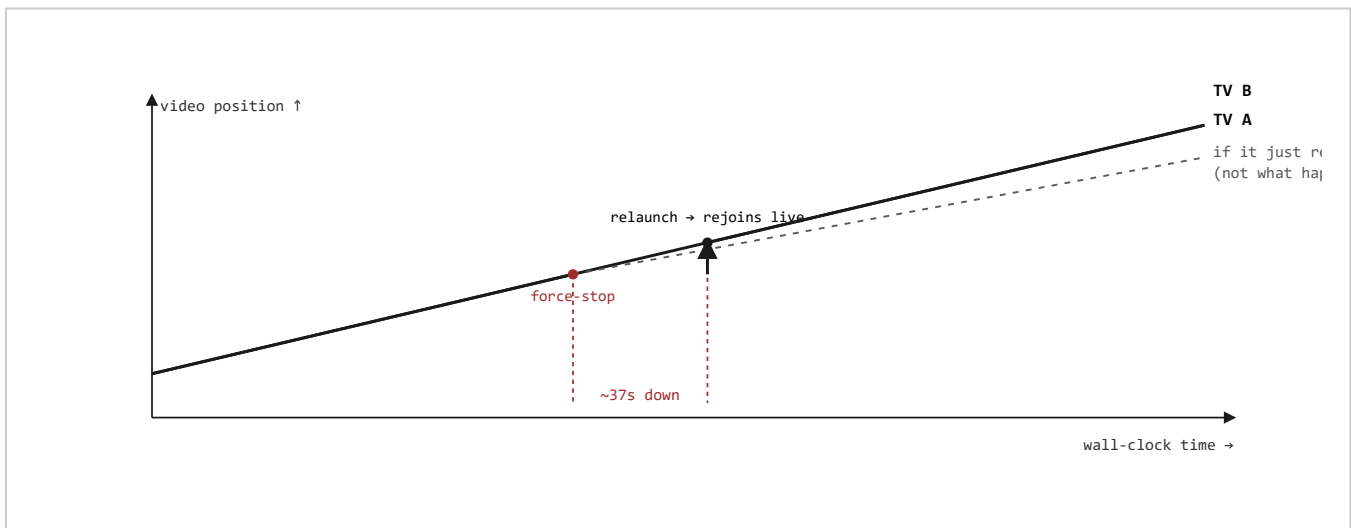


Fig. 4 — TV B doesn't continue from where it paused (grey dashed line) and doesn't restart at zero — it re-fetches the shared clock and jumps to the live position. Measured after recovery: TV A at 07:14 / common clock 92:52, TV B at 07:21 / common clock 92:59 — seven seconds apart, matching the real gap between the two checks, both drifting under 0.2s. No per-device position is ever persisted; this is a live rejoin, not a personal resume.

6. Drawbacks of this prototype

Everything above is proven to work. What follows is what it doesn't yet address, grouped by what breaks first if this were pointed at real traffic.

Reliability

- One Node process holds the entire shared clock in memory — restart it, and every device silently re-baselines to a new start time.
- No health checks, no redundancy, nothing watching the sync server itself.
- `serverClockOffset` only refreshes every 60s, so per-device clock drift can accumulate up to that window.

Security

- Login is a hardcoded client-side check (an email/password compared directly in JS) — no real backend authentication, trivially bypassed.
- The `/session` endpoint is open to anyone who can reach the host: no auth, no rate limiting, no per-device identity.
- A Pusher app key and a hardcoded auth-endpoint URL live in client source in a related prototype file.

Scale

- One global timeline: every connected device joins the exact same program. There's no concept of separate channels, rooms, or facilities.
- Polling-based resync doesn't scale as a push mechanism and caps how quickly a correction can reach a device.
- The clock lives in one process's memory — you can't run a second instance without the two disagreeing.

Product gaps

- One hardcoded video, forever. No playlist, no schedule, no ad breaks, no rotation.
- No admin control plane — nobody can pause, skip, or restart the "channel"; it just runs from whenever the process started.
- No fleet visibility — no dashboard of which devices are connected or how well they're tracking; everything lives in each device's own console log.
- The Android networking fix (`10.0.2.2`) only works in the emulator, not on real hardware.

7. Path to production — what the backend needs to provide

The prototype answers "does the sync math work." A production version needs a real backend behind it. The shape of that change:

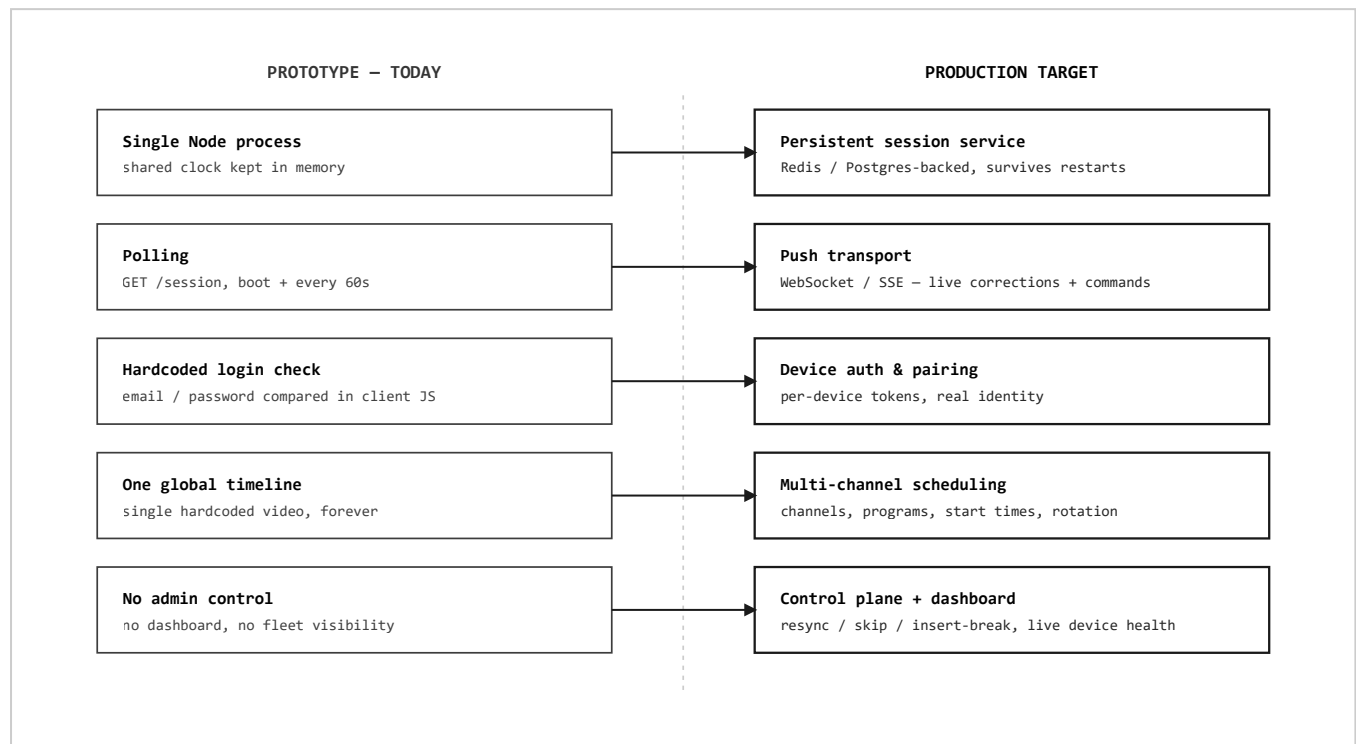


Fig. 5 — Each row is one prototype shortcut and the production capability that replaces it. None of the right column exists yet.

Backend capabilities needed, and why

Capability	What it replaces	Why it's needed
Persistent session/clock service	In-memory <code>sync-server1.cjs</code>	A restart today silently re-baselines every connected device to a new start time. A real store (Redis/Postgres) makes the timeline survive process restarts and deploys.
Push transport (WebSocket/SSE)	Polling <code>GET /session</code> every 60s	Removes the staleness window and is the only way to deliver live commands — a passive poll can't tell a device to pause or skip.
Device auth & pairing	Hardcoded client-side login check	Every device needs its own revocable identity, not a shared password baked into the bundle. (This mirrors the <code>tv_<uuid></code> per-device pattern already used elsewhere in this codebase's Pusher integration.)
Multi-channel scheduling	One hardcoded video URL	Real deployments need more than one program running, and a way to define what plays when — a channel/program model backed by a datastore, not a source-code constant.
Admin control plane + fleet dashboard	Nothing	Operators currently have no way to intervene or even see how a device is doing beyond its own console log. Needed to catch a struggling device before a viewer does.
Production content infrastructure	A public demo HLS stream	Real content needs transcoding and an ABR ladder behind a CDN. Worth checking whether the existing <code>phen-video-service</code> / <code>phen-video-transcoding-lambda-service</code> projects in this workspace already cover this before building new infrastructure.
Horizontal scalability	A single Node process	The clock needs to live in shared storage so more than one backend instance can serve it — today, exactly one process can exist at a time.

8. Reference

Platform-specific networking

Environment	Resolved host	Why
Browser (dev/testing)	<code>window.location.hostname</code>	Works from whatever machine served the page.
Android emulator	<code>10.0.2.2</code>	The emulator's fixed alias back to the host machine.
Physical Android device	– not handled	Needs the host's real LAN IP; not resolved automatically today.

Tunable constants

Constant	Value	Meaning
<code>SYNC_THRESHOLD</code>	<code>0.2s</code>	Drift below this is left alone.
<code>RATE_CORRECT_THRESHOLD</code>	<code>2.0s</code>	Ceiling for rate-nudge correction before a hard seek.
<code>RATE_FAST</code> / <code>RATE_SLOW</code>	<code>1.05×</code> / <code>0.95×</code>	Playback-rate nudge inside the rate-correct band.
Periodic sync tick	<code>1000ms</code>	Steady-state drift recheck interval.
Clock resync interval	<code>60000ms</code>	How often the server clock offset is refreshed.

Reflects the state of `VideoSyncTimelinePoc.js`, `sync-server1.cjs`, and `src/utils/syncServer.js` after the drift-correction and Android-networking fixes, verified manually across two Android TV emulators (API 36) plus one force-stop/recovery test.